

FACAMP · TRATAMENTO E ARMAZENAMENTO DA INFORMAÇÃO

Torra & Terra

Do requisito ao deploy

Como construímos um e-commerce com integridade garantida pelo banco de dados.

Prof. Nivaldo T. Marcusso

Como trabalhamos

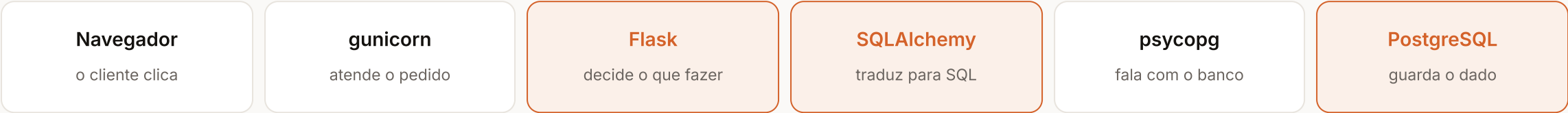
Oito etapas, uma revisão entre cada uma, um commit por etapa.



Nenhuma etapa começou antes da anterior ser revisada. O histórico do Git conta essa ordem — são commits que contam a construção, não um único “projeto pronto”.

Como o site funciona por dentro

O caminho que um clique percorre — do navegador do cliente até o dado guardado no banco.



O QUE É CADA PEÇA, EM UMA FRASE

gunicorn	Python	É o porteiro . Fica escutando a internet e entrega cada acesso para o Python. Sem ele, o site aguentaria uma pessoa por vez.
Flask	Python	É o cérebro . Decide o que responder em cada endereço, e é onde moram as regras — “sem estoque, não vende”.
SQLAlchemy	Python	É o tradutor . A gente escreve em Python, ele converte para SQL, que é a língua que o banco entende.
psycopg	Python	É o cabo . Leva o comando SQL até o PostgreSQL pela rede e traz a resposta de volta.
PostgreSQL	SQL	É o cofre . Guarda tudo e, principalmente, recusa qualquer dado que quebre as regras.
Jinja	HTML	É o molde da página. Pega o dado que veio do banco e encaixa no HTML que o cliente vê.

A interface é HTML e CSS escritos à mão — sem Bootstrap, sem Tailwind, sem JavaScript nenhum.

Ferramentas

O que usamos em cada etapa, e para quê.

MODELAGEM E BANCO

- **Mermaid** — DER versionado junto com o código
- **psql** — aplicar o schema e tirar evidências
- **EXPLAIN** — conferir se os índices são usados
- **pg_dump / pg_restore** — backup e o teste de restauração

DESENVOLVIMENTO

- **VS Code** — edição e terminal
- **Git e GitHub** — commits, um por etapa
- **Claude Code** — par de programação
- **python-dotenv** — segredos fora do código

QUALIDADE E OPERAÇÃO

- **pytest** — 24 testes contra Postgres real
- **OWASP ZAP** — varredura de segurança
- **Railway** — aplicação e banco em cloud
- **Playwright** — capturas automatizadas

Etapa 0 — Requisitos antes de qualquer código

A tentação é começar pelo schema. Começamos pelo problema.

A PERGUNTA QUE ORIENTOU TUDO

Como concluir um pedido com integridade garantida pelo banco, e não pela boa vontade da aplicação?

- **6 requisitos funcionais** — cada um com critério de aceite
- **8 requisitos não funcionais** — integridade, segurança, desempenho, recuperação
- **Backlog priorizado** — o que entra no MVP e o que fica como evolução
- **Fluxo do processo** — com os pontos de validação marcados

Decidimos aqui que o carrinho seria sessão, e não tabela: ele é processo, não entidade — existe só enquanto a compra não fecha.

Etapas 1 e 2 — O modelo e as constraints

Escrevemos o DDL à mão. `db.create_all()` não é chamado em lugar nenhum.

A DECISÃO DE MODELAGEM CENTRAL

A moagem mora no item, não no produto.

O cliente escolhe na compra. É esse atributo que faz `itens_pedido` ser entidade de verdade, e não tabela de ligação.

11

CHECK

4

FOREIGN KEY

5

PRIMARY KEY

3

UNIQUE

A validação do formulário protege a experiência; a constraint protege o dado. Se amanhã alguém inserir por script ou por API, a regra continua valendo — `pontuacao_sca BETWEEN 80 AND 100` é regra de negócio, não de interface.

Etapa 6 — Finalizar a compra sem deixar rastro pela metade

O coração do trabalho. Ou grava tudo, ou não grava absolutamente nada.

ABRE A OPERAÇÃO

- |
- | — PRIMEIRO — reserva e confere, sem gravar nada ainda
 - | | — tranca a linha de cada café, para ninguém mexer ao mesmo tempo
 - | | — esse café ainda existe?
 - | | — tem estoque para a quantidade pedida?
- |
- | — DEPOIS — só agora grava
 - | | — cria o pedido e cada item, com o preço do dia congelado
 - | | — desconta o estoque
- |
- | — CONFIRMA . ou DESFAZ TUDO, se qualquer coisa falhar

POR QUE TRANCAR SEMPRE NA MESMA ORDEM

Se duas compras simultâneas trancassem os mesmos cafés em ordens opostas, cada uma ficaria esperando a outra soltar — e **as duas travariam para sempre**.
Trancando sempre na mesma ordem, isso não acontece.

TRÊS BARREIRAS, UMA ATRÁS DA OUTRA

1. o navegador não deixa digitar mais que o estoque · 2. o site confere de novo, com o café já reservado · 3. o banco recusa qualquer estoque negativo, mesmo que as duas primeiras falhem.

Etapa 7 — Os testes, e o que eles pegaram

24 testes contra um PostgreSQL real. Não SQLite.

POR QUE NÃO SQLITE

O `SELECT ... FOR UPDATE` e as constraints `CHECK` são justamente o que precisa ser testado — e o SQLite trata os dois de forma diferente. Testar contra outro banco daria confiança falsa.

UM TESTE NOSSO ESTAVA ERRADO

Inseríamos a moagem 'EXTRAFINA' esperando que o `CHECK` recusasse. O banco recusou — mas **por tamanho**: a coluna é `VARCHAR(6)` e a palavra tem 9 caracteres. O dado era barrado, mas pela regra errada.

- Compra normal grava pedido, itens e baixa estoque
- Estoque insuficiente faz rollback completo
- Produto inexistente não finaliza o pedido
- Preço, estoque, SCA e moagem inválidos são recusados pelo banco
- Senha nunca é gravada em texto puro
- Mesmo café em duas moagens vira duas linhas
- Preço congelado sobrevive a um reajuste

Tres problemas que so apareceram em producao

Funcionar na máquina do desenvolvedor não é funcionar.

1

A aplicação usava superusuário

O provedor entrega a conexão como postgres, que é superusuário do container. Isso violava um requisito do próprio material. Criamos o papel torra_app, sem SUPERUSER. Verificado: DROP TABLE é negado.

2

O pg_dump local era velho demais

Cliente 17.5 contra servidor 18.6 — o pg_dump se recusa a dumpar um servidor mais novo que ele. Rodamos o backup de dentro do container do banco, onde as ferramentas são da versão exata.

3

O deploy automático não dispara

O repositório pertence a outra conta do GitHub, e o Railway respondeu: “no one in the project has access to it”. Depende do dono do repositório autorizar. Enquanto isso, o deploy é manual e funciona.

Testamos a segurança da loja

Passamos um programa que procura falhas — o OWASP ZAP. Ele apontou 11 problemas. Oito eram reais, e a gente corrigiu.

O PROBLEMA SÉRIO: PEDIDO FEITO SEM O CLIENTE QUERER

Imagine que o cliente está logado na nossa loja e abre outro site, malicioso, em outra aba. Esse site consegue mandar um “finalizar compra” para a nossa loja — e o navegador anexa a identificação do cliente automaticamente, porque ele não sabe distinguir de onde veio o pedido. **A compra sairia de verdade.**

COMO RESOLVEMOS

Cada formulário nosso passou a carregar uma **senha escondida**, que muda a cada sessão e só o nosso site conhece. Se o pedido chegar sem ela, é recusado. E o navegador foi instruído a não enviar a identificação do cliente quando o pedido vier de outro site.

OS OUTROS SEIS, EM LINGUAGEM SIMPLES

- **Só trafegar por conexão segura** — a identificação do cliente nunca vai por canal aberto
- **Dizer ao navegador o que pode carregar** — nada de script de fora
- **Proibir que a loja seja embutida em outro site** — evita botão falso por cima do real
- **Exigir sempre HTTPS** — mesmo se alguém digitar o endereço sem ele
- **Não deixar o navegador “adivinhar” tipo de arquivo**
- **Não guardar em cache** a página de quem está logado

*A loja não usa **nenhum JavaScript**. Isso permitiu bloquear scripts por completo — e é a defesa mais forte possível contra código injetado. Esta página de slides também não usa, pelo mesmo motivo.*

0 projeto em números

5

tabelas

23 constraints e 4 índices

24

testes

todos passando

12

cafés

em 4 regiões produtoras

27

commits

um por etapa da construção

O QUE FICOU DE FORA, DE PROPÓSITO

Pagamento real e frete · Área administrativa · Relatórios de venda · **Controle de lote e data de torra**, a evolução mais natural do domínio · **Carrinho entre dispositivos**, que é exatamente onde um Redis passaria a fazer sentido.

Demonstração

nivaldo.felipefurlan.com.br

- Catálogo e filtro por região
- Escolha da moagem — a decisão de modelagem, na tela
- Checkout: o pedido gravado e o estoque baixando
-
- O SELECT provando que nada foi gravado

O Chapada Geisha está com estoque 1, de propósito.